

Week 02: Tabular Q-Learning

Discovering Optimal Action-Values from Experience

Reinforcement Learning & Robot Learning Core Curriculum

Presented by **DGIST ISL Team**

DGIST Intelligent Systems and Learning Laboratory

2026.08.21

Core Concepts: Q-Table · Temporal-Difference Error · Bellman Optimality · ϵ -Greedy Policy

From Domain Heuristics to Value Discovery

Can an agent autonomously discover optimal policies without human priors?

The Action-Value Function $Q(s, a)$

Definition: $Q(s, a) = \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t r_t \mid s_0 = s, a_0 = a, \pi \right]$

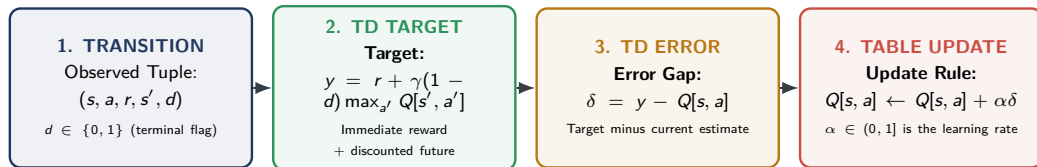
- "If I am in state s and execute action a , what total return do I expect afterwards?"
- **Tabular Form:** 2D array of shape $|\mathcal{S}| \times |\mathcal{A}|$.
- Initialized to zeros (0.00) = neutral initial evidence.

LineWorld Q-Table Structure

State s	Left ($a = 0$)	Right ($a = 1$)	Best Action $\arg \max_a Q$
$s = 0$ (Start)	0.00 $\rightarrow$ 0.70	0.00 $\rightarrow$ 0.77	Right
$s = 1$	0.00 $\rightarrow$ 0.70	0.00 $\rightarrow$ 0.86	Right
$s = 2$	0.00 $\rightarrow$ 0.77	0.00 $\rightarrow$ 0.96	Right
$s = 3$	0.00 $\rightarrow$ 0.86	0.00 $\rightarrow$ 1.00	Right (Immediate Goal)
$s = 4$ (Goal)	Terminal state (no action chosen)		Terminated

The Temporal-Difference (TD) Learning Mechanism

Bootstrapping: updating an estimate from an immediate reward and an existing future guess



Non-Terminal Case ($d = 0$)

Target $y = r + \gamma \max_{a'} Q[s', a']$. Value propagates backwards from goal one step at a time.

Terminal Case ($d = 1$)

Target $y = r$ at true task termination. An external time-limit truncation usually keeps the bootstrap term.

Concrete Walkthrough: TD Arithmetic in LineWorld

Hand-traceable example: tracing values from initial state to goal ($(\alpha = 0.5, \gamma = 0.9)$)

Step 1: Goal Step ($s = 3 \xrightarrow{\text{right}} 4$)

Initial Table: $Q[3, \text{right}] = 0.00, r = +1.00, d = \text{True}$

Terminal Target: $y = r = 1.00$

TD Error: $\delta = y - Q[3, \text{right}] = 1.00 - 0.00 = +1.00$

Update: $Q[3, \text{right}] \leftarrow 0.00 + 0.5(1.00) = +0.50$

Discovery: The agent now knows state 3 has high value!

Step 2: Value Propagation ($s = 2 \xrightarrow{\text{right}} 3$)

Current Table: $Q[2, \text{right}] = 0.00, \max Q[3] = 0.50, r = -0.01$

Target: $y = -0.01 + 0.9(0.50) = +0.44$

TD Error: $\delta = 0.44 - 0.00 = +0.44$

Update: $Q[2, \text{right}] \leftarrow 0.00 + 0.5(0.44) = +0.22$

Insight: Value flows from state 3 back into state 2 even with negative step cost $r = -0.01$!

Exploration vs. Exploitation: ϵ -Greedy Policy

Balancing the discovery of new paths with the exploitation of known high-value actions

The Dilemma

- **Pure Exploitation** ($\epsilon = 0$): Always pick $\arg \max_a Q(s, a)$. May miss actions made unattractive by early estimates or tie-breaking.
- **Pure Exploration** ($\epsilon = 1$): Random actions; does not deliberately exploit discovered paths.

ϵ -Greedy Action Selection

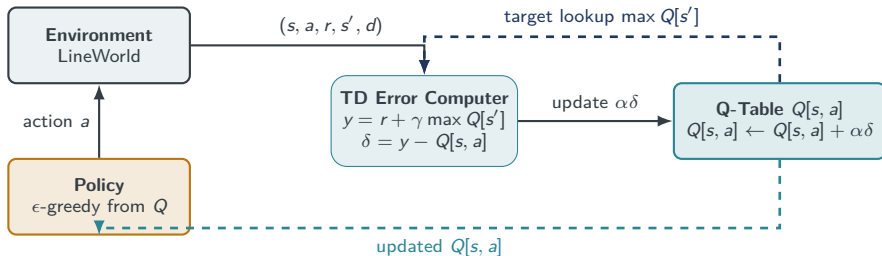
$$a_t = \begin{cases} \text{random action} \in \mathcal{A}, & \text{with prob } \epsilon \\ \arg \max_a Q[s_t, a], & \text{with prob } 1 - \epsilon \end{cases}$$

Training vs Evaluation Discipline

- **During Training:** Use $\epsilon > 0$ (e.g. $\epsilon = 0.20$) so the agent discovers the entire state-action graph.
- **During Evaluation:** Set $\epsilon = 0$ (Pure Greedy).
- Evaluating with $\epsilon > 0$ injects artificial noise into policy benchmark metrics.
- Evaluate what the agent *learned*, not how it explored!

System Architecture: The Tabular Q-Learning Loop

How exploration, environment feedback, and table updates form an autonomous learning loop



Code Lens: Tabular Q-Learning in Python

Implementation patterns in examples/week-02

Q-Table Data Structure

Python 2D List

```
q_table = [[0.0, 0.0] for _ in range(5)]  
def update(s, a, r, s_next, term):  
    if term:  
        target = r  
    else:  
        target = r + gamma * max(q_table[s_next])  
    delta = target - q_table[s][a]  
    q_table[s][a] += alpha * delta
```

Empirical Evaluation Benchmark

Policy	Avg Return	Success Rate
Random Policy	+0.52	65%
Always Right (Prior)	+0.97	100%
Trained Q-Learner	+0.97	100%

What We Can Claim

Under this seed and evaluation protocol, Q-learning matched the known always-right reference without a hardcoded directional rule.

Failure Modes in Tabular Q-Learning

Common conceptual and algorithmic pitfalls in tabular implementations

1. TRUNCATION BOOTSTRAP BUG

For an external time limit, setting $y = r$ discards continuation value. True task termination—goal or failure—drops the bootstrap term.

2. ZERO EXPLORATION TRAP ($\epsilon = 0$)

With $\epsilon = 0$, action coverage depends on initial values and tie-breaking; useful alternatives may never be sampled.

3. EVALUATING EXPLORATORY POLICIES

For this greedy Q-policy, report $\epsilon = 0$ evaluation separately from exploratory behavior; do not mix the protocols.

4. LEARNING RATE INSTABILITY

$\alpha = 1$ replaces an estimate with one sampled target; it does not imply instant global convergence. Stochastic targets retain variance, and standard convergence conditions use decaying step sizes.

Robotics Bridge: The Limits of Tabular Scaling

Why tabular lookups collapse on real physical robotic systems

The Curse of Dimensionality

- 7-DOF robot arm with joint angles discretized into 100 bins each:

Table Rows = $100^7 = 10^{14}$ discrete states!

- Storing 10^{14} floats requires ≈ 800 Terabytes of RAM.
- The robot would need millions of years of rollouts to visit every cell even once!

The Bridge to Function Approximation

- Tabular cells do not generalize: visiting state (1.000) gives zero knowledge about state (1.001).
- **Parametric Approximation (Weeks 3–4):** Replace $Q[s, a]$ table with neural network $Q_\theta(s, a)$.
- Parameterized weights generalize value estimates across continuous physical state spaces.

Week 02 Quiz: Core Concepts & Action-Values

Self-check: verify fundamental understanding of tabular Q-learning

Q1. Meaning of $Q[s, a]$

What quantity does each cell in a Q-table estimate?

- A. The immediate reward received from executing action a .
- B. The total expected cumulative return from state s , taking action a , and acting optimally thereafter.
- C. The probability of transitioning to the goal state.

Answer: B $\implies$ Action-value formulation.

Q2. Temporal-Difference Error δ

What does the TD error $\delta = y - Q[s, a]$ represent?

- A. The gradient of the policy return.
- B. The discrepancy between the newly bootstrapped target y and the prior value estimate $Q[s, a]$.
- C. The distance to the goal state.

Answer: B $\implies$ Core learning signal.

Week 02 Quiz: TD Update Arithmetic

Verify hand-computations for tabular Q-learning updates

Problem 1: Goal Terminal Update

Given: $\alpha = 0.5$, $Q[3, \text{right}] = 0.00$, $r = +1.00$, $d = \text{True}$

Calculate: Target y , TD Error δ , and updated $Q[3, \text{right}]$.

Solution:

- $d = \text{True} \implies y = r = 1.00$
- $\delta = 1.00 - 0.00 = +1.00$
- $Q[3, \text{right}] \leftarrow 0.00 + 0.5(1.00) = +0.50$

Problem 2: Non-Terminal Value Propagation

Given: $\alpha = 0.5$, $\gamma = 0.9$, $Q[2, \text{right}] = 0.00$, $r = -0.01$

Next state values: $\max Q[3] = 0.50$, $d = \text{False}$

Calculate: Target y , TD Error δ , and updated $Q[2, \text{right}]$.

Solution:

- $y = -0.01 + 0.9(0.50) = +0.44$
- $\delta = 0.44 - 0.00 = +0.44$
- $Q[2, \text{right}] \leftarrow 0.00 + 0.5(0.44) = +0.22$

Week 02 Quiz: Interpretation & Practical Traps

Key practical questions every RL practitioner must master

Q3. Value Increase with Negative Reward

How can $Q[2, \text{right}]$ rise with $r = -0.01$?

Because the future discounted value $\gamma \max Q[3] = +0.45$ outweighs the small step cost -0.01 , yielding positive target $+0.44$.

Q4. Why Initial Table Starts at Zero

Why start with all zeros in the Q-table?

Zero is convenient, not universally neutral: it can be optimistic or pessimistic depending on the possible returns and also exposes tie-breaking behavior.

Q5. Training vs Evaluation Policy

Why evaluate with $\epsilon = 0$ instead of $\epsilon = 0.2$?

$\epsilon = 0$ isolates the learned greedy policy. Report exploratory behavior separately when it is also operationally relevant.

Q6. Tabular LineWorld Claim Limits

What does solving LineWorld show?

This implementation learned the known best path in one toy MDP under the stated protocol; it does not establish broader RL or robotics performance.